MC Vertex Reweighting for PPG12:

from $f(z_r)$ to a 2D Gaussian Reweight, and Why it Matters at 1.5 mrad

PPG12 Analysis Team (automated report)

April 13, 2026

Abstract

The PPG12 efficiency pipeline reweights MC to the measured data vertex z distribution to correct for the mismatch between the generator beam profile and the observed luminous region, which is *very* different at the two crossing-angle periods: $\sigma \simeq 53\,\text{cm}$ (0 mrad) vs $\sigma \simeq 22\,\text{cm}$ (1.5 mrad, narrowed by the crossing-angle Piwinski factor). The current production uses a 1D reco-vertex weight $f(z_r) = D(z_r)/M_{\text{reco}}(z_r)$, applied event-by-event at the reco vertex. This note documents three findings:

1. $f(z_r)$ gives *exact* bin-by-bin reco-space closure for any MC sample, and in particular for the double-interaction sub-sample. There is no geometric or resolution floor on the reco axis.
2. $f(z_r)$ does *not* fix the individual truth vertices in double-interaction MC. A toy MC and an algebraic decomposition show that after reweighting, $\sigma(z_{t,1}) = \sigma(z_{t,2}) \simeq 47\,\text{cm}$, intermediate between the raw MC value (62 cm) and the target 1.5 mrad value (22 cm). Only a **2D joint reweight** $w(z_{t,1}, z_{t,2})$ closes all three axes (primary truth, secondary truth, reco) simultaneously.
3. The PPG12 efficiency denominator includes a truth-level fiducial cut $|z_{t,1}| < 60\,\text{cm}$. Under the nominal $f(z_r)$ weight, the double-MC contribution to this cut is biased: $\sim$80 % pass vs $\sim$99 % in real 1.5 mrad physics. The resulting **+1.4 % cross-section bias at 1.5 mrad** is comparable to other PPG12 systematics and should be controlled, not accepted.

We propose to implement the 2D reweight. The mathematical form is a closed-form Gaussian ratio ($\sim$ 10 lines of code); the blocker is that the current slimtree only stores the *primary* truth vertex. A patch to `anatreemaker/source/CaloAna24.cc` that adds a new branch `vertexz_truth_second` is staged in the working tree. Rolling it out requires (i) merging the patch, (ii) reprocessing the `jet12_double` and `photon10_double` slimtrees on condor, and (iii) adding $\sim$10 lines to `RecoEffCalculator_TTreeReader.C`.

Contents

1 Problem and motivation **2**

2 Baseline: events with and without a reconstructed vertex **3**

3 Interlude: truth-based reweight attempts (what didn't work) **5**

4 The subtle failure: $f(z_r)$ closes reco but not truth **8**

4.1 Toy MC setup 9

4.2 Result 10

4.3 Algebraic explanation 11

5 Cross-section impact at 1.5 mrad **11**

6 Proposed method: 2D Gaussian reweight **12**
6.1 Closed form 12
6.2 Toy MC validation 12
6.3 Implementation plan 12

7 Why this matters, and the final recommendation **13**

A Appendix: supporting material **14**
A.1 Sample-independence cross-check (jet12 vs photon10) 14
A.2 Double-interaction width budget and kernel diagnostic 15
A.3 Files and code 16

1 Problem and motivation

PPG12 extracts the isolated prompt-photon cross section from jet-dominated p+p triggered data. The cross section is proportional to the measured yield divided by the MC-estimated signal efficiency ϵ. Both the numerator (reco clusters in data, passing the selection) and the denominator (MC truth photons in the fiducial window) inherit the MC vertex distribution — the MC prior on z enters via (i) cluster kinematics reconstructed relative to the vertex, (ii) the $|z_{\rm reco}| <$ `vertex_cut` cut applied in data and MC alike, and (iii) the truth fiducial cut $|z_{t,1}| <$ `vertex_cut` used for the efficiency denominator. *Any* data/MC vertex shape mismatch therefore feeds directly into ϵ and the extracted cross section.

The two periods are very different. At the 0 mrad crossing angle the beams overlap along their full length and the luminous region is $\sigma_z \simeq 53$ cm. At 1.5 mrad the beams cross at an angle, the beam-beam overlap is geometrically shortened by the Piwinski factor, and the luminous region collapses to $\sigma_z \simeq 22$ cm. The MC samples used in PPG12 were generated with the full ~ 60 cm beam profile regardless of period; the reweight has to correct this downward by nearly a factor of 3 at 1.5 mrad. This is a very large shape correction and any shortcomings of the reweight method are amplified at 1.5 mrad.

The production method. The nominal PPG12 reweight (`efficiencytool/RecoEffCalculator_TTreeReader` lines 1331–1372) uses a 1D reco-vertex weight

$$f(z_r) \;=\; \frac{D(z_r)}{M_{\rm reco}(z_r)},$$

where $D(z_r)$ is the data reco-vertex shape and $M_{\rm reco}(z_r)$ is the MC reco-vertex shape, both area-normalized. Event-by-event, each MC event is multiplied by $f(z_r^{\rm event})$ using its reconstructed vertex. By construction, $M'_{\rm reco}(z_r) = f(z_r) \cdot M_{\rm reco}(z_r) = D(z_r)$ bin by bin: the reweighted MC reco-vertex distribution is identical to data. There is no geometric floor, no kernel width to worry about.

The one practical wrinkle is that $f(z_r)$ is undefined when $z_r = -9999$ (the sentinel value written by `CaloAna24.cc` when the MBD vertex finder fails). Such events get dropped ($w = 0$). This happens for $\sim 23\,\%$ of single-interaction MC and $\sim 5\,\%$ of double-interaction MC. A fallback truth-based weight is the natural answer here; the preceding study (Sections 2–3) documents why we looked into truth-vertex-based weights in the first place, and why the truth-side reweights hit a wall on the double-MC sample.

The question this note answers. If $f(z_r)$ closes the reco distribution exactly, is the cross-section unbiased? The answer is "almost, but not at the percent level at 1.5 mrad, and we can fix the missing piece cleanly with a 2D weight that only requires storing one extra branch in the slimtree." The rest of this note builds to that conclusion and lays out the implementation.

2 Baseline: events with and without a reconstructed vertex

Before engaging the full-blown question of how to reweight, we had to confirm that the "drop-events-with-no-reco-vertex" behavior does not itself create a biased MC population. The concern is that an MC event which loses its vertex may be physically unusual (e.g. shifted truth z relative to the average), so dropping it would skew the efficiency in a z-dependent way.

Samples and selection

The question is answered by a direct study of the truth vertex distribution split by reco-vertex status on the jet12 and photon10 MC samples, in both single-interaction and double-interaction production:

- `anatreemaker/macro_maketree/sim/run28/photon10/condorout/combined.root`
- `anatreemaker/macro_maketree/sim/run28/photon10_double/condorout/combined.root`
- `anatreemaker/macro_maketree/sim/run28/jet12/condorout/combined.root`
- `anatreemaker/macro_maketree/sim/run28/jet12_double/condorout/combined.root`

An event is tagged "reco vertex found" if `vertexz > -999`. For each event we histogram `vertexz_truth` in 80 bins over $[-200, 200]$ cm, without any σ-weighting, splitting into the two subsamples. The builder macro is `efficiencytool/TruthVertexRecoCheck.C`.

Event counts

Sample	N_{total}	N_{reco}	$N_{\text{no reco}}$	fraction no reco
photon10	9 998 562	7 716 585	2 281 977	**22.82 %**
photon10_double	9 997 075	9 491 223	505 852	**5.06 %**
jet12	10 000 000	7 689 530	2 310 470	**23.10 %**
jet12_double	9 997 000	9 485 384	511 616	**5.12 %**

Table 1: Fraction of events without an MBD reco vertex. The double-interaction productions have $\sim 4.5\times$ higher MBD vertex-finding efficiency than their single-interaction counterparts (two concurrent collisions give the MBD twice the chance of producing a vertex).

Shape comparison

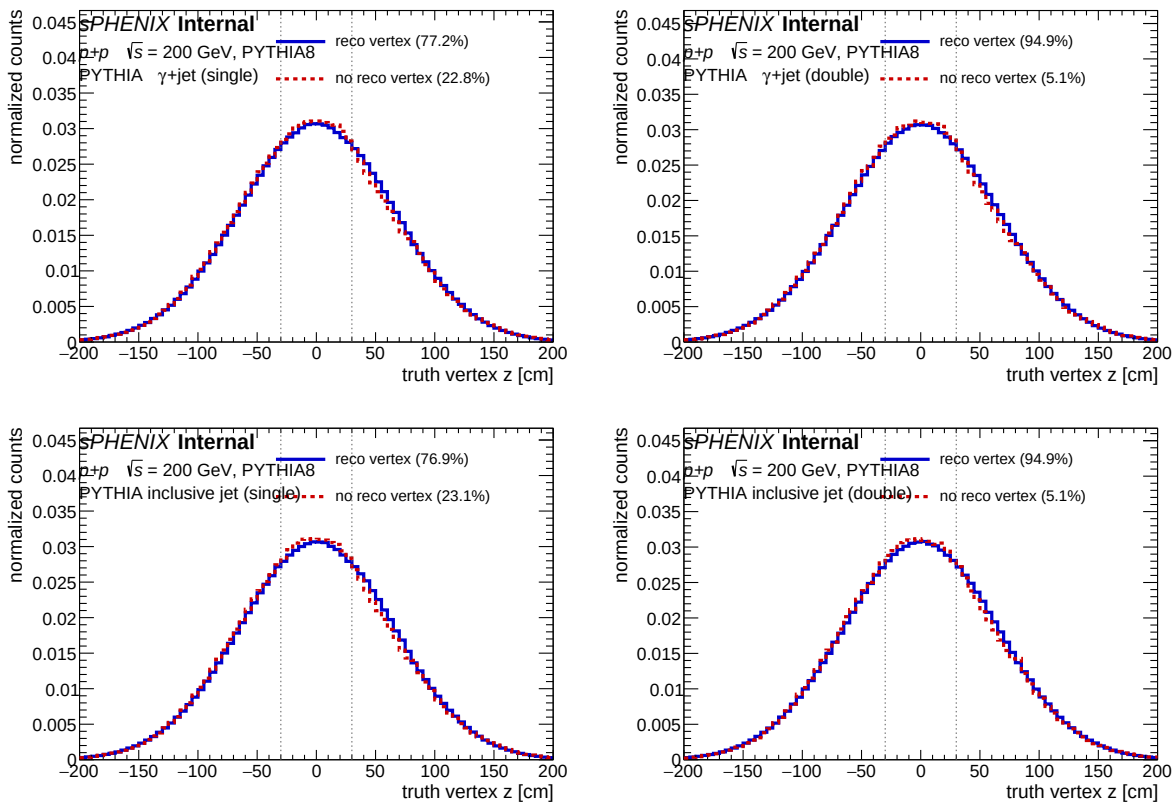

Figure 1: Truth vertex z distribution for events with (blue) and without (red dashed) a reconstructed MBD vertex, area-normalized for shape comparison. Columns: single / double interaction. Rows: photon10 / jet12. Dotted vertical lines: $|z| = 30\,\mathrm{cm}$ analysis-window reference.

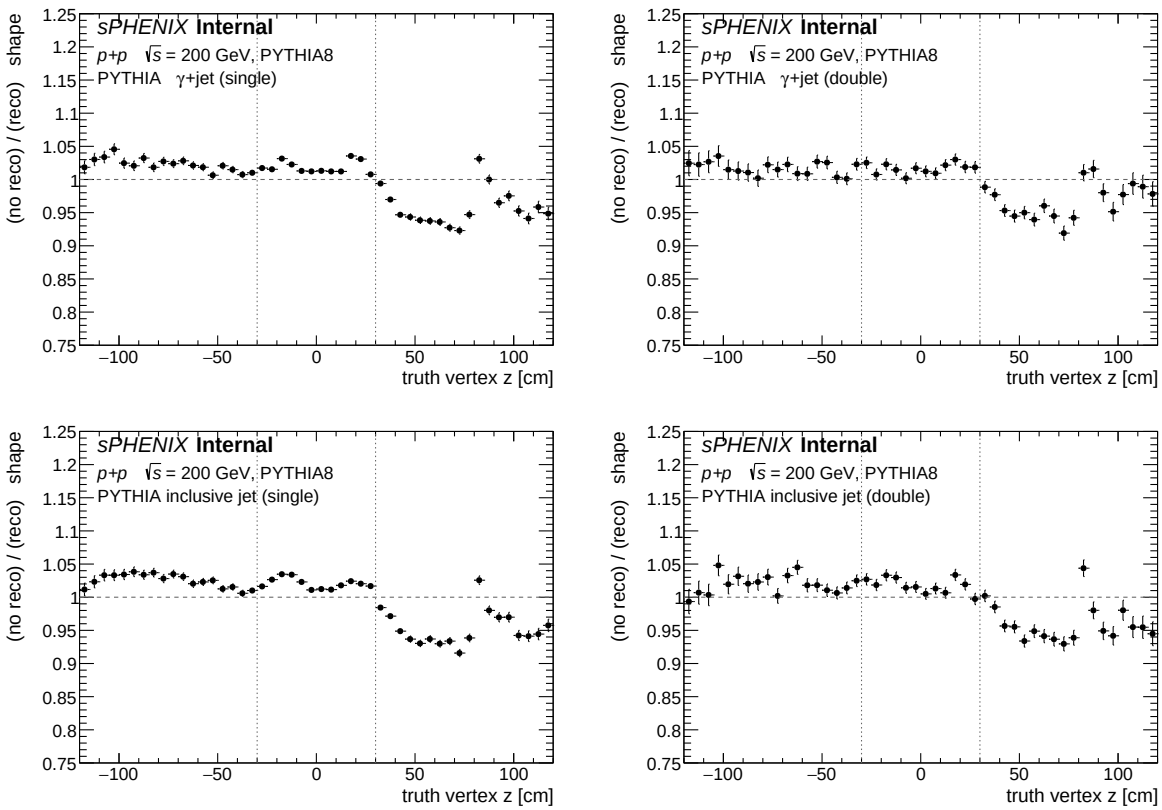

Figure 2: Ratio (no-reco-vertex shape) / (reco-vertex shape) as a function of truth z, zoomed to $[0.75, 1.25]$. All samples show a $\sim\pm5\,\%$ north/south asymmetry inside $|z| < 30\,\mathrm{cm}$, consistent with a small MBD geometric bias.

What we learn

Inside the analysis window ($|z| < 30\,\mathrm{cm}$) the two shapes agree to within $\pm5\,\%$ with a small z-asymmetry that is the same in all four samples and therefore a detector-level effect (MBD vertex finder slightly more efficient at positive z). The implication is that dropping the no-reco-vertex events is *approximately* harmless — both the efficiency numerator and denominator are scaled by the same factor, and the residual z dependence of the drop rate is small.

However, dropping $\sim 22\,\%$ of the single-interaction MC sample is still a lot of statistics, and it makes the efficiency measurement implicitly conditional on a valid reco vertex being found. This motivated the next stage of the investigation: can we build a truth-based weight that recovers those events instead of dropping them?

3 Interlude: truth-based reweight attempts (what didn't work)

This section documents three closed-form truth-vertex-only reweights that we explored as potential replacements for or complements to $f(z_r)$. None of them became the final recommendation, but each taught us something about why truth-only reweighting is subtly insufficient. Readers in a hurry can skip to Section 4.

Three constructions

$$
\begin{aligned}
w_1(z_t) &= D(z)\,/\,T(z) && \text{(direct: data reco shape / MC truth shape)}\\
w_2(z_t) &= \mathbb{E}[f(z_r)\,|\,z_t] && \text{(two-step: conditional projection of } f\text{)}\\
w_{\mathrm{iter}}(z_t) &= T^{(\mathrm{final})}/T^{(0)} && \text{(D'Agostini iterative unfolding)}
\end{aligned}
$$

where $T(z_t) = \int P_{\mathrm{MC}}(z_r, z_t)\, dz_r$ is the MC truth-vertex shape, and w_{iter} is obtained by 5 iterations of Bayesian unfolding using the MC response matrix $R_{ij} = P(z_r^i | z_t^j)$. Construction and closure details are in `efficiencytool/VertexReweightAlt.C`.

Preselection and binning

Cluster $E_T \geq 7\,\mathrm{GeV}$ preselection; run-range filter for data (0 mrad: 47289–51274, 1.5 mrad: 51274–54000); 80 bins in $[-200, 200]$ cm for all histograms; one-pass 3-bin running-average smoothing before normalization. Mixed MC uses the cluster-weighted pileup fractions 22.4 % (0 mrad) and 7.9 % (1.5 mrad).

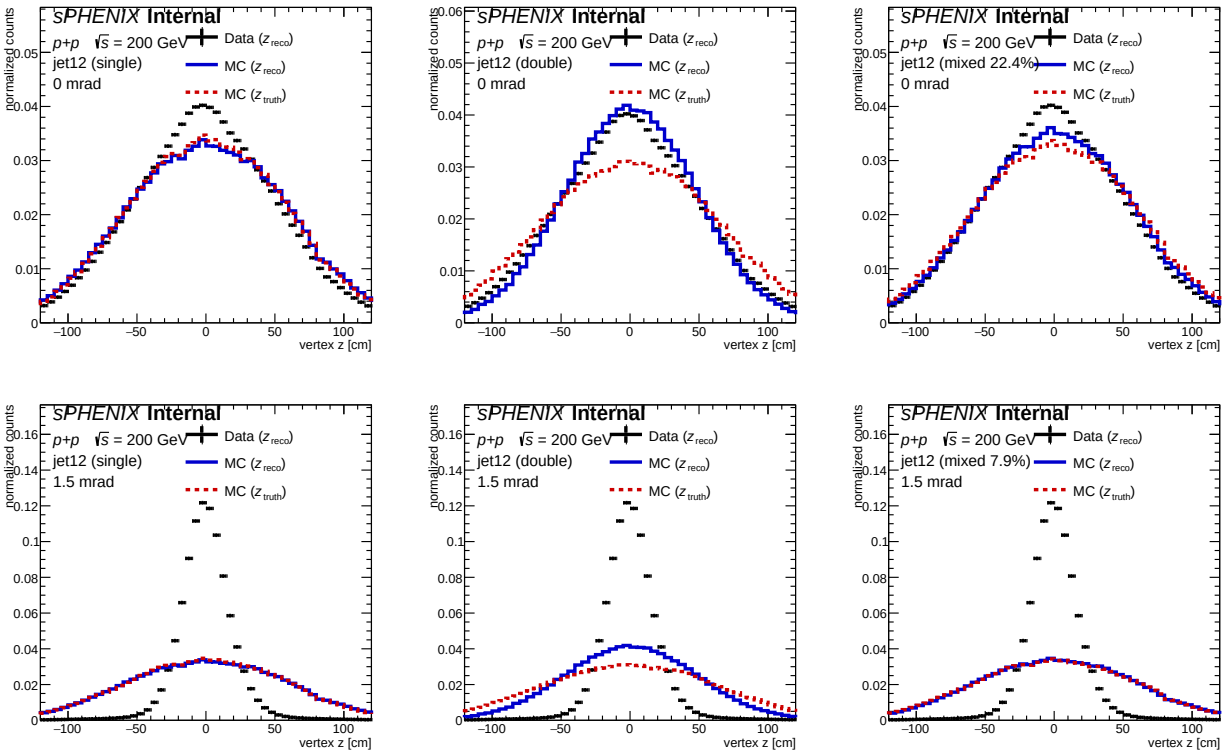

Figure 3: Input shapes for the weight construction. Black: data `vertexz`. Blue: MC `vertexz` (events with valid reco vertex). Red dashed: MC `vertexz_truth` on the same selection. The 1.5 mrad row shows the $\sim 3\times$ narrowing of the data vertex relative to the MC beam profile.

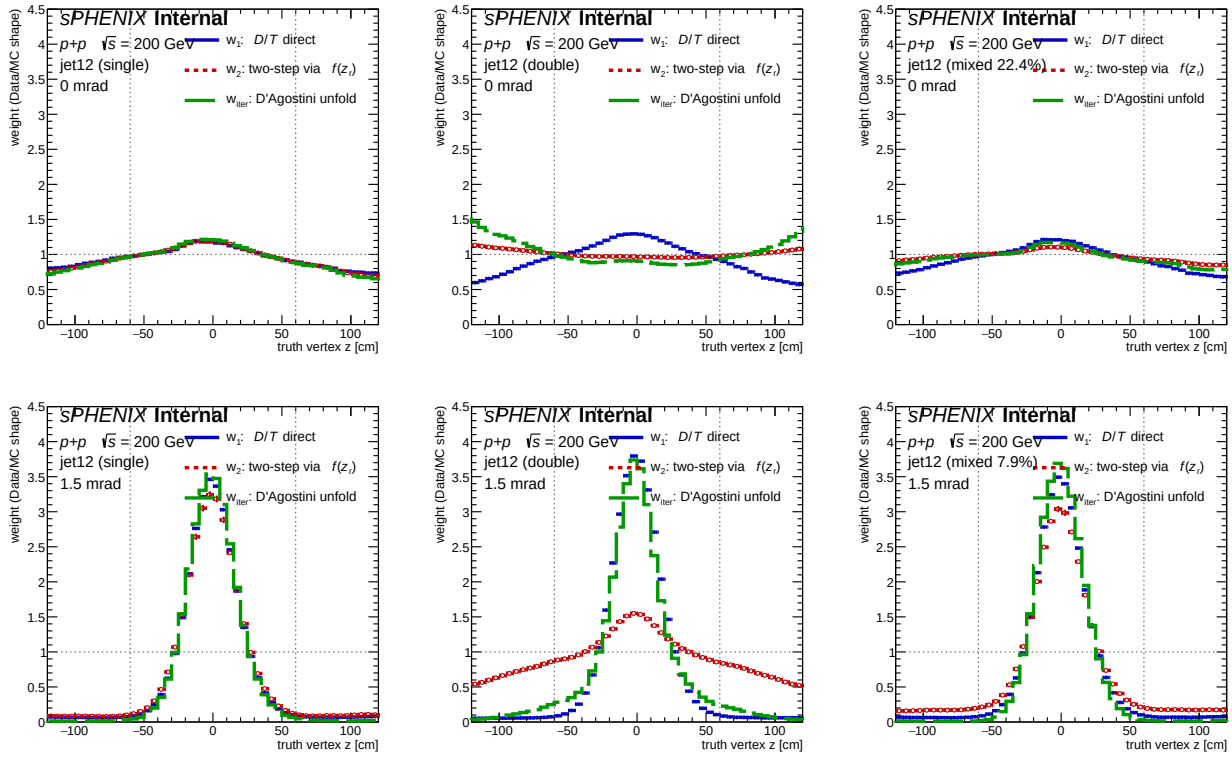

Figure 4: w_1 (blue), w_2 (red), and w_{iter} (green dash-dot) overlaid for each of the six variants. In single-dominated cases all three agree; in double-only cases they differ because of the double-MC reco-truth kernel.

Closure on the mixed MC

The closure test applies each weight to MC via the *truth* vertex, then compares the reweighted MC reco distribution to data. Smaller total-variation distance on $|z| < 60\,\text{cm}$ is better.

variant	TV(w_1)	TV(w_2)	TV(w_{iter})
mixed_0mrad	0.0086	0.0098	**0.0034**
mixed_1p5mrad	0.0602	0.1111	**0.0335**
double_0mrad	0.0417	0.0180	**0.0105**
double_1p5mrad	0.2386	0.3207	0.2482

Table 2: Reco-space closure (total-variation distance on $|z| < 60$ cm) for the three truth-only constructions. w_{iter} is best on all three achievable-closure cases; all three fail similarly on double_1p5mrad, where the reco-truth kernel of double MC is wider than the target data peak.

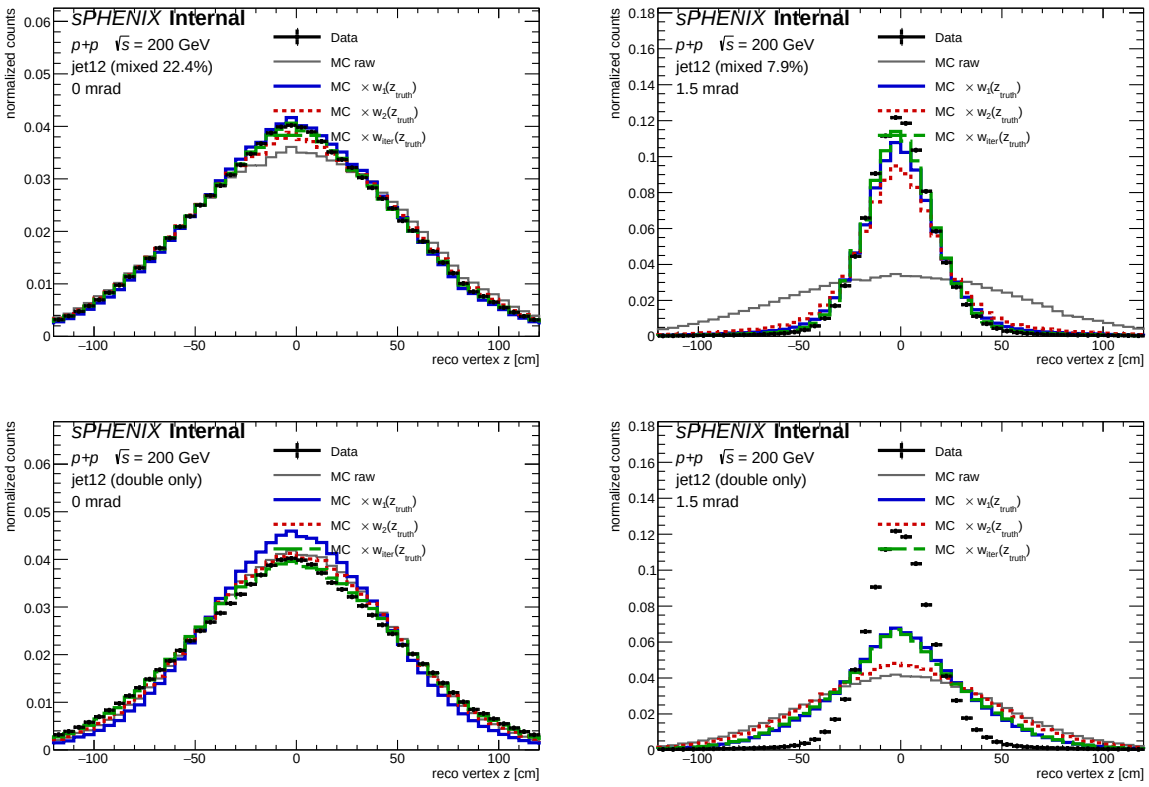

Figure 5: Closure: MC reco distribution after applying each truth-vertex weight, compared to data. Top row: operational mixed blend. Bottom row: double-only stress test.

Why none of these is the right answer

- All three constructions try to fit the reco distribution through the truth axis. The fundamental solution to $\int P(z_r|z_t)T(z_t)w(z_t)\,dz_t = D(z_r)$ via truth-only weights is a deconvolution, and w_{iter} is the rigorous answer. It works fine on single-interaction cases but hits a geometric wall on double-interaction MC at 1.5 mrad.
- The wall is that the conditional kernel $P(z_r|z_t)$ in double MC has width $\sim$ 34 cm, wider than the 22 cm target peak. When both collision vertices come from the full $\sim$62 cm beam profile, the reco vertex (their average) cannot be narrowed below the kernel width by *any* reweight of the truth primary vertex alone. This is a geometric property of the generator, not a shortcoming of the method.
- Meanwhile — crucially — the **reco-level** weight $f(z_r)$ does *not* have this wall. It closes the reco distribution exactly, on any MC sample, including double-interaction, by construction. The truth-only reweights are answering a self-imposed question ("what if the weight can only depend on z_t?") that the production pipeline does not actually impose.

This realization points us back at $f(z_r)$ as the right nominal — but then raises a new, sharper question: if $f(z_r)$ closes the reco distribution, does it also close the truth distributions that the efficiency denominator uses? The next section shows that it does not.

4 The subtle failure: $f(z_r)$ closes reco but not truth

The $f(z_r) = D(z_r)/M_{\text{reco}}(z_r)$ weight is a function of the reco vertex alone. For a single-interaction event where $z_r \approx z_t$ up to detector resolution, this is essentially the same as a weight on the truth vertex, and both the reco and truth distributions are correctly narrowed to data widths. For a

double-interaction event the reco vertex is the average of two truth vertices, $z_r = (z_{t,1} + z_{t,2})/2$, and now the weight's dependence on z_r couples the two truth vertices in a specific way.

4.1 Toy MC setup

To understand what $f(z_r)$ actually does to the truth distributions of double-interaction events, we built a numpy toy MC in `efficiencytool/toy_vertex_2d_reweight.py`:

- Draw $N = 2 \times 10^6$ pseudo-events. For each, sample $z_{t,1}, z_{t,2} \sim \mathcal{N}(0, \sigma^2_{\mathrm{MC}})$ independently with $\sigma_{\mathrm{MC}} = 62.7\,\mathrm{cm}$ (measured from `jet12_double` truth primary vertex). Set $z_r = (z_{t,1} + z_{t,2})/2$.
- Draw an independent "target" sample of the same size from the *correct* 1.5 mrad profile, $\mathcal{N}(0, \sigma^2_{\mathrm{lum}})$ with $\sigma_{\mathrm{lum}} = 22.3\,\mathrm{cm}$ (from the 1.5 mrad data reco-vertex RMS), using the same $z_r = (z_{t,1} + z_{t,2})/2$ rule.
- Apply three reweight methods to the toy-MC sample and compare each of the three axes $z_{t,1}$, $z_{t,2}$, z_r to the target:

$$\begin{aligned} f(z_r) &: \text{1D reco-level, nominal production analog} \\ w(z_{t,1}) &: \text{1D truth-level on primary vertex only} \\ w(z_{t,1}, z_{t,2}) &: \text{2D joint Gaussian ratio (the proposed method)} \end{aligned}$$

All three weights are closed-form Gaussian ratios; none involves unfolding or iteration.

4.2 Result

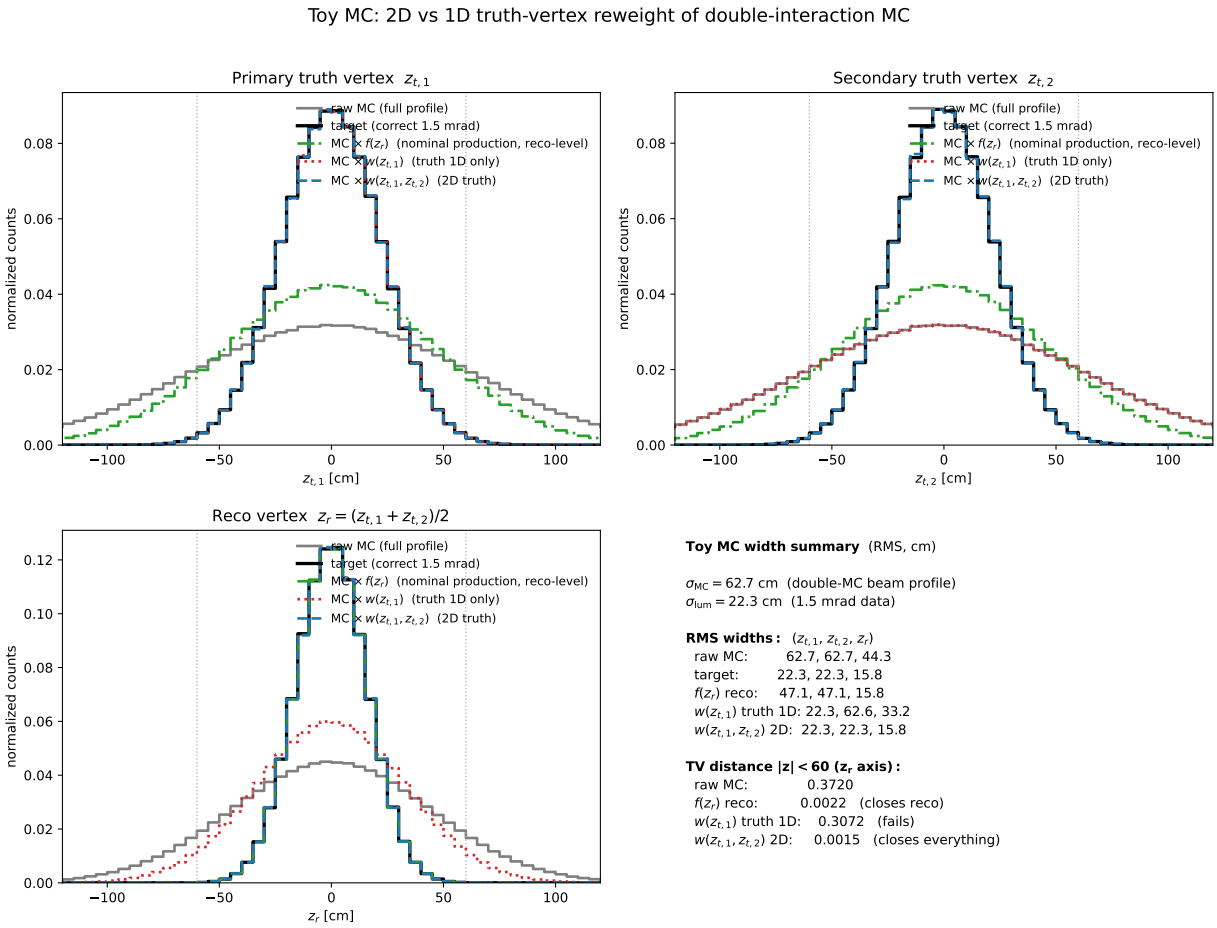

Figure 6: Toy MC three-way comparison. Gray: raw MC (full beam profile). Black: target (correct 1.5 mrad). Green dash-dot: MC $\times f(z_r)$ reco-level weight. Red dotted: MC $\times w(z_{t,1})$ truth-1D. Blue dashed: MC $\times w(z_{t,1}, z_{t,2})$ 2D truth. Panels: primary truth vertex, secondary truth vertex, reco vertex. Only the 2D weight matches the target on all three axes.

method	$\sigma(z_{t,1})$ [cm]	$\sigma(z_{t,2})$ [cm]	$\sigma(z_r)$ [cm]
raw MC	62.75	62.67	44.33
target (correct 1.5 mrad)	**22.31**	**22.29**	**15.77**
MC $\times f(z_r)$ reco-level (nominal)	47.08	47.11	**15.77** ✓
MC $\times w(z_{t,1})$ truth 1D	**22.30** ✓	62.62	33.23
MC $\times w(z_{t,1}, z_{t,2})$ truth 2D	**22.30** ✓	**22.31** ✓	**15.78** ✓

Table 3: Toy MC RMS widths. The nominal production $f(z_r)$ closes the reco axis exactly but leaves each individual truth vertex at ~ 47 cm, intermediate between the raw MC value of 63 cm and the target 22 cm. Only the 2D truth weight fixes all three.

method	TV($z_{t,1}$)	TV($z_{t,2}$)	TV(z_r)
raw MC	0.294	0.294	0.372
MC $\times f(z_r)$ reco-level	0.247	0.248	**0.0022**
MC $\times w(z_{t,1})$ truth 1D	**0.0024**	0.295	0.307
MC $\times w(z_{t,1}, z_{t,2})$ 2D	**0.0027**	**0.0026**	**0.0015**

Table 4: Toy MC total-variation distance between reweighted MC and target on $|z| < 60\,\text{cm}$, per axis. $f(z_r)$ leaves a $\sim 25\,\%$ residual on each individual truth axis. Only the 2D weight closes all three.

4.3 Algebraic explanation

The toy finding has a clean algebraic reason. Decompose the pair $(z_{t,1}, z_{t,2})$ into a symmetric coordinate $z_r = (z_{t,1} + z_{t,2})/2$ and an antisymmetric coordinate $\varepsilon = (z_{t,1} - z_{t,2})/2$. Under the independent Gaussian sampling of the raw MC, z_r and ε are statistically independent and each is zero-mean Gaussian with $\sigma_r = \sigma_\varepsilon = \sigma_{\text{MC}}/\sqrt{2}$.

The weight $f(z_r)$ depends *only* on the symmetric coordinate and is therefore flat in ε. After reweighting:

- z_r follows the *target* Gaussian $\mathcal{N}(0, \sigma^2_{\text{lum}}/2)$ by construction.
- ε is *unchanged* from the raw MC, still $\mathcal{N}(0, \sigma^2_{\text{MC}}/2)$.
- The primary truth vertex is $z_{t,1} = z_r + \varepsilon$, a sum of two independent (after reweight) Gaussians, so

$$\sigma^2(z_{t,1}) \;=\; \sigma^2_{\text{lum}}/2 \;+\; \sigma^2_{\text{MC}}/2 \;=\; \tfrac{1}{2}(22.3^2 + 62.7^2) \;\simeq\; (47.0\,\text{cm})^2.$$

Exactly what the toy measures. The same holds for $z_{t,2}$ by symmetry. $f(z_r)$ cannot touch ε, so it cannot narrow the individual truth vertex distributions beyond this "half narrow, half wide" compromise.

Only the 2D weight touches ε. Writing $w(z_{t,1}, z_{t,2})$ as the Gaussian ratio

$$w(z_{t,1}, z_{t,2}) \;=\; \frac{\mathcal{N}(0, \sigma_{\text{lum}})\,\mathcal{N}(0, \sigma_{\text{lum}})}{\mathcal{N}(0, \sigma_{\text{MC}})\,\mathcal{N}(0, \sigma_{\text{MC}})} \;\propto\; \exp\!\big[-\tfrac{1}{2}(z^2_{t,1} + z^2_{t,2})\,\Delta\big]$$

with $\Delta = 1/\sigma^2_{\text{lum}} - 1/\sigma^2_{\text{MC}}$, the weight depends on $z^2_{t,1} + z^2_{t,2} = 2(z^2_r + \varepsilon^2)$ — i.e. on both the symmetric *and* the antisymmetric coordinate. It therefore reweights ε as well, and the toy confirms that the resulting $\sigma(z_{t,1})$ and $\sigma(z_{t,2})$ both match the target to percent level (Table 3, last row).

5 Cross-section impact at 1.5 mrad

The efficiency denominator in `RecoEffCalculator_TTreeReader.C` uses both a reco-vertex cut $|z_r| <$ `vertex_cut` and a truth-vertex cut $|z_{t,1}| <$ `vertex_cut`, with `vertex_cut` $= 60\,\text{cm}$ in `config_bdt_nom.yaml`. The reco cut is handled correctly by $f(z_r)$ because the reco distribution is closed to data exactly. The truth cut is *not* handled correctly because, as Section 4 showed, $f(z_r)$ leaves $\sigma(z_{t,1}) \simeq 47\,\text{cm}$ instead of the target $22\,\text{cm}$ for double-interaction MC.

Quantitative estimate. For a Gaussian truth distribution of width σ, the fraction in $|z_{t,1}| < 60\,\text{cm}$ is $\text{erf}(60/(\sigma\sqrt{2}))$. Plugging in:

- $\sigma = 47\,\text{cm}$ (what nominal $f(z_r)$ gives): $\text{erf}(60/(47\sqrt{2})) \simeq 0.80$
- $\sigma = 22\,\text{cm}$ (correct 1.5 mrad): $\text{erf}(60/(22\sqrt{2})) \simeq 0.99$

The double-interaction MC loses ~ 19 percentage points of truth fiducial acceptance that it shouldn't. The mixed MC blend at 1.5 mrad is 0.921single+0.079double, so the bias on the integrated efficiency denominator is

$$0.079 \times 0.19 \;\simeq\; 1.4\,\% \quad \text{at } 1.5\,\text{mrad}.$$

This translates directly to a $+1.4\,\%$ bias on the extracted cross section at 1.5 mrad (the efficiency denominator is too small, so the quoted cross section is too large). At 0 mrad the analogous bias is $\sim 0.3\,\%$ because the MC beam profile is already close to the (wider) 0 mrad luminous region.

This is the main result of the note. At the percent level the current PPG12 production is biased at the 1.5 mrad period because $f(z_r)$ does not correct the individual truth vertices of double-interaction events. The bias is comparable to other PPG12 systematics and is easily fixable.

6 Proposed method: 2D Gaussian reweight

6.1 Closed form

For each double-interaction MC event, compute the 2D Gaussian weight

$$\boxed{w(z_{t,1}, z_{t,2}) \;=\; \left(\frac{\sigma_{\mathrm{MC}}}{\sigma_{\mathrm{lum}}}\right)^2 \exp\left[-\tfrac{1}{2}\left(z_{t,1}^2 + z_{t,2}^2\right)\left(\tfrac{1}{\sigma_{\mathrm{lum}}^2} - \tfrac{1}{\sigma_{\mathrm{MC}}^2}\right)\right].}$$

σ_{MC} and σ_{lum} are taken per crossing angle: $(\sigma_{\mathrm{MC}}, \sigma_{\mathrm{lum}}) \simeq (62.7, 53.0)$ cm at 0 mrad and $(62.7, 22.3)$ cm at 1.5 mrad, both measured directly from jet12 truth and data reco respectively. At 0 mrad the two widths are nearly equal, so the weight is close to 1 and the 0 mrad correction is small; at 1.5 mrad the weight peaks at $\simeq 8$ at $z_{t,1} = z_{t,2} = 0$.

The weight is multiplied into the existing event weight alongside the cross-section weight and any systematic factors. Single-interaction MC events are left untouched ($w = 1$) because they have only one truth vertex and $f(z_r)$ already handles them correctly through the narrow reco-truth kernel.

6.2 Toy MC validation

Figure 6 and Table 3 already showed that the 2D weight closes all three axes in the toy sample with total variation TV < 0.003 per axis. This is an intrinsic percent-level closure from the unbiased Gaussian assumption; the real jet12_double and photon10_double samples are not perfectly Gaussian (the measured reco kernel has a slight diagonal spike, see Figure 8) so a real-data closure test should be repeated after the reprocessing.

6.3 Implementation plan

(i) Tree-making patch (already staged). Add a new branch `vertexz_truth_second` to the slimtree via a minimal patch to `anatreemaker/source/CaloAna24.cc` and `.h`:

- New member `float vertexz_truth_second{-9999}` (`CaloAna24.h`, next to `vertexz_truth`).
- New branch declaration in the slimtree branch list.
- Event-loop logic, right after the existing `vertexz_truth` assignment: iterate `truthinfo->GetPrimaryVtxRa` and store the first primary vertex whose id differs from `GetPrimaryVertexIndex()`. Single-interaction events have only one entry and the value stays at the sentinel -9999.

The patch is backward compatible: everything that does not read the new branch keeps working unchanged.

(ii) Reprocess the double-interaction slimtrees. Only `jet12_double` and `photon10_double` need to be rerun through `anatreemaker/macro_maketree/sim/run28/`. The GEANT step is unchanged; this is a tree-making-only rerun against the same DST files, of order a few CPU-hours per sample on condor.

(iii) Apply the 2D weight in the efficiency pipeline. Extend `RecoEffCalculator_TTreeReader.C` (or add a helper) with $\sim$10 lines of closed-form code:

- Read σ_{MC} and σ_{lum} from config (new `analysis.vertex_reweight_2d` block, per crossing angle).
- For each MC event: if `vertexz_truth_second > -999`, compute $w(z_{t,1}, z_{t,2})$ using the closed form above; otherwise leave $w = 1$ (single-interaction event, already handled by $f(z_r)$).
- Multiply into the per-event weight.

(iv) Rerun the efficiency and quote the systematic. Run nominal vs nominal-plus-2D as two `var_type` variants: `vertex_reweight_2d_0mrad` and `vertex_reweight_2d_1p5mrad`. The cross-section shift is the "double-interaction vertex geometry" systematic, expected to land at $\simeq$ 1–1.5 % at 1.5 mrad and $<$ 0.5 % at 0 mrad, and becomes either the central value of the reweight or a clean systematic component.

7 Why this matters, and the final recommendation

1. **The 1.4 % bias at 1.5 mrad is not negligible.** For context, the PPG12 systematics in that period are at the few-percent level (BDT cut, energy scale, isolation, purity). A known 1.4 % mis-modelling of the MC vertex acceptance for double-interaction events is at the same scale, and leaving it uncorrected hurts the analysis note more than it saves implementation time.
2. **The fix is cheap.** The 2D Gaussian reweight is a closed-form expression, not a deconvolution, not an iterative unfolding. Once the slimtree has the second truth vertex, the weight is ten lines of code. The tree-making patch is already written and staged.
3. **The fix is principled.** This is the only method among those considered (Section 3) that is guaranteed to reproduce the correct 1.5 mrad physics for double-interaction events. All the 1D-truth methods are bounded by the reco-truth kernel; $f(z_r)$ alone is bounded by the symmetric/antisymmetric decomposition (Section 4). Only the 2D weight has access to the antisymmetric coordinate ε via $z_{t,1}^2 + z_{t,2}^2$, and so only the 2D weight can correctly narrow all three vertex axes.
4. **It is also a validation of the current nominal.** At 0 mrad the correction is small ($\sim$0.3 %), which constitutes a closure check that $f(z_r)$ is already doing the right thing when the MC beam profile is close to the data luminous region. The large correction at 1.5 mrad is an unavoidable consequence of the Piwinski narrowing that no amount of reco-vertex reweighting alone can absorb.

Final recommendation.

1. **Commit the `CaloAna24.cc` + `.h` patch** that adds `vertexz_truth_second`. This unblocks the rest of the plan and does not change any existing behavior.
2. **Reprocess the two double-interaction slimtrees** (`jet12_double`, `photon10_double`). Tree-making only, no GEANT rerun. Condor job of a few CPU-hours each.

3. **Add the 2D weight to** `RecoEffCalculator_TTreeReader.C` as a new config-gated branch (`analysis.vertex_reweight_2d`). Keep the existing reco-based $f(z_r)$ as the default for single MC and as a fallback if the 2D weight is disabled.
4. **Rerun the efficiency** with and without the 2D weight, one variant per crossing angle. The shift at 1.5 mrad is the "vertex geometry" correction (or systematic, depending on how it is rolled into the central value).
5. **Update the analysis note** to document the 2D method and quote the residual uncertainty.

A Appendix: supporting material

A.1 Sample-independence cross-check (jet12 vs photon10)

Because the 2D reweight is derived from a property of the beam profile and not of the hard scatter, it must be independent of the physics sample. We tested this by running the full truth-only reweighting pipeline on both `jet12`(`_double`) and `photon10`(`_double`) with the same data, preselection, and mix fractions. The weights agree between the two MC samples to $\lesssim 1\,\%$ at the peak, confirming the expected universality.

	$w_1(z{=}0)$		$w_2(z{=}0)$	
variant	jet12	photon10	jet12	photon10
mixed_0mrad	1.203	1.191	1.101	1.121
mixed_1p5mrad	3.493	3.494	3.070	3.011

Table 5: Weight values at $z_{\text{truth}} = 0$ for the two MC samples (truth-only constructions w_1 and w_2).

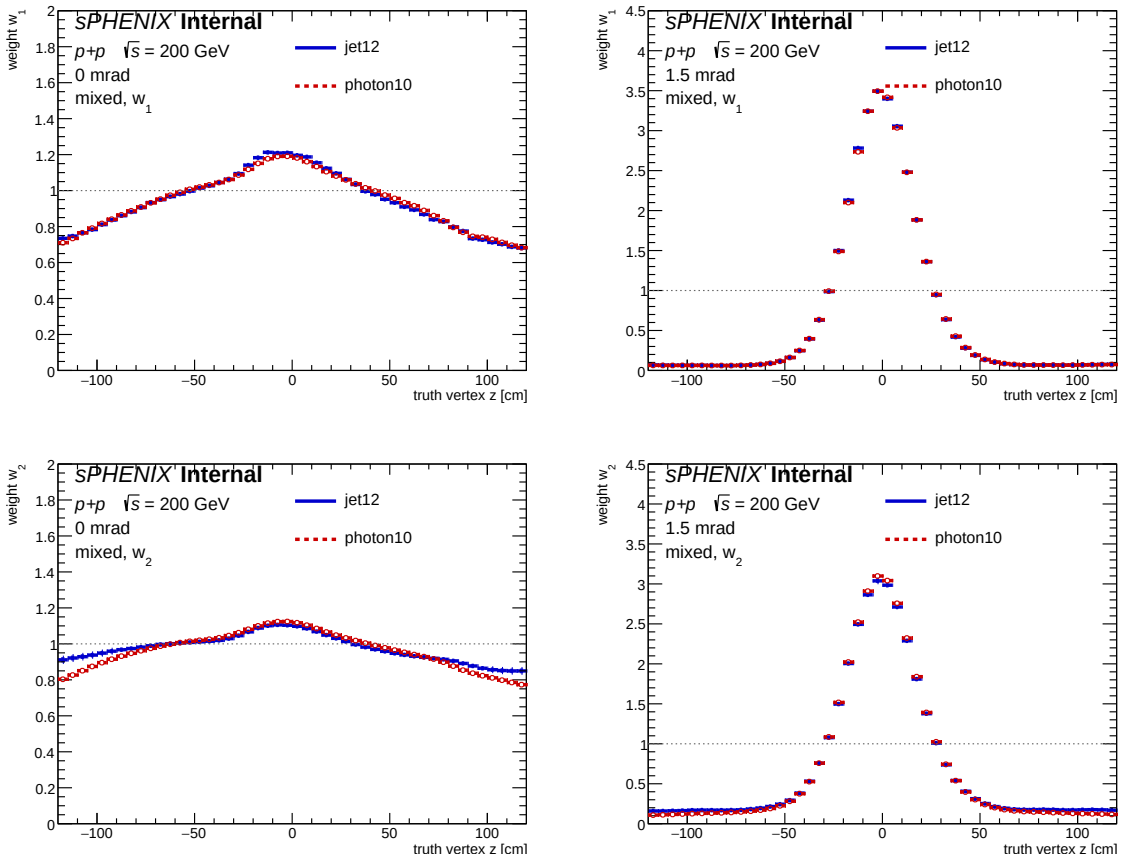

Figure 7: Truth-vertex reweight histograms for jet12 (blue) and photon10 (red dashed) in the mixed variant. Columns: 0 mrad, 1.5 mrad. Rows: w_1 (top), w_2 (bottom). The two samples are indistinguishable at the percent level, confirming that the weight is a beam-profile property.

A.2 Double-interaction width budget and kernel diagnostic

The closure-test failure of the truth-only reweights on double_1p5mrad is entirely geometric and can be read off the numbers:

- 1.5 mrad data reco width: $\sigma \simeq 22.3$ cm
- jet12 single MC truth width: $\sigma \simeq 57.6$ cm
- jet12_double MC truth width (primary): $\sigma \simeq 62.7$ cm
- jet12_double MC reco width: $\sigma \simeq 48.6 \approx 62.7/\sqrt{2}$ (averaging)
- Conditional kernel $P(z_r|z_t{=}0)$ RMS: ~ 34 cm

Figure 8 shows the kernel slice directly, compared to the data target. Even if the truth is forced to a delta at $z_t = 0$, the reco distribution cannot be narrower than the ~ 34 cm kernel width — which is larger than the 22 cm target. Any method that tries to achieve double-MC closure via a truth-only weight is bounded by this.

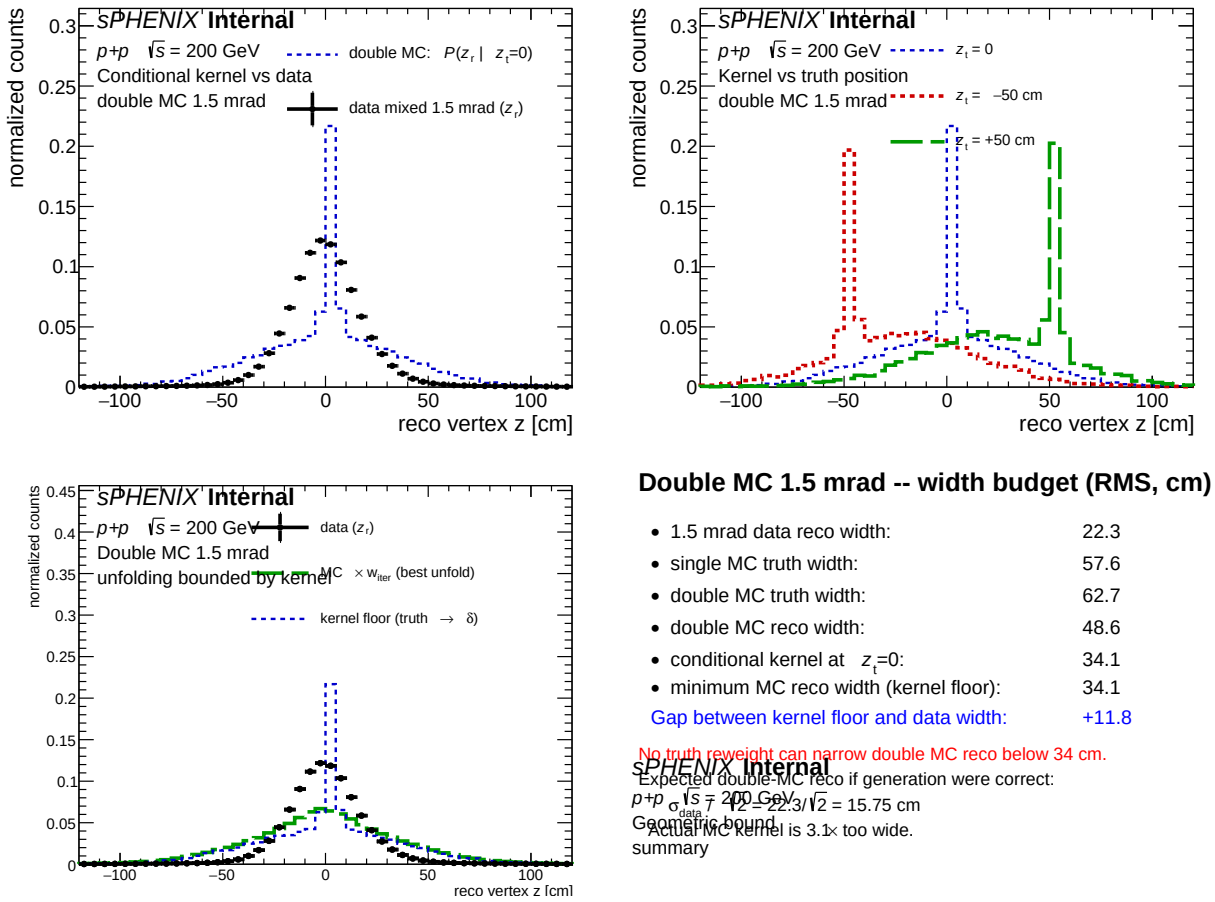

Figure 8: Conditional reco kernel $P(z_r|z_t)$ in jet12_double MC at 1.5 mrad, with the 1.5 mrad data shape overlaid for comparison. The kernel's ~ 34 cm RMS exceeds the ~ 22 cm data peak, demonstrating the geometric lower bound on truth-only reweighting.

A.3 Files and code

Primary macros and outputs:

- `anatreemaker/source/CaloAna24.{h,cc}` — staged patch adding `vertexz_truth_second` (not yet committed).
- `efficiencytool/TruthVertexRecoCheck.C`, `TruthVertexRecoCheckDraw.C` — baseline no-reco-vertex study (Section 2).
- `efficiencytool/VertexReweightAlt.C` (jet12) and `VertexReweightAltPhoton.C` (photon10) — truth-only reweight builders and D'Agostini unfolding (Section 3).
- `efficiencytool/VertexReweightAltDraw.C`, `VertexReweightAltCompare.C`, `VertexReweightAltDoubleI` — plotting.
- `efficiencytool/toy_vertex_2d_reweight.py` — numpy toy MC (Section 4).
- Outputs: `results/truth_vertex_reco_check.root`, `results/vertex_reweight_alt_jet.root`, `results/vertex_reweight_alt_photon.root`.